Below is a **complete Java Collections Framework guide** with:

✅ **All internal data structures (data types) used**
✅ **Implementation details (how it works internally)**
✅ **Differences between collection classes**
✅ **When to use what (real-time decision)**
✅ **Java example code for every individual collection class**

---

1) Internal Data Types (Data Structures) used in Java Collections Framework

Java Collections are implemented internally using these **core DS/data types**:

Internal Data Type (DS)	Meaning	Collections That Use It Internally	Why used / Benefit
Dynamic Array	Resizable array	ArrayList, Vector, Stack, CopyOnWriteArrayList	Fast access by index O(1)
Singly Linked List	Node → next	old HashMap bins (before Java 8), internal iterators	simple chaining
Doubly Linked List	Node has prev + next	LinkedList	Fast insert/delete in middle
Hash Table (Hashing + Buckets)	hash(key) → index	HashMap, HashSet, Hashtable, ConcurrentHashMap	Fast search/insert average O(1)
Bucket Array	Array of buckets	HashMap, HashSet	store collisions
Tree (Red-Black Tree)	Self balancing BST	TreeMap, TreeSet, Java 8+ HashMap bins (if collision high)	Sorted + O(log n)
Heap (Binary Heap)	Priority structure	PriorityQueue	always gives min/max element
Deque (double ended queue)	add/remove both ends	ArrayDeque, LinkedList	faster queue/stack replacement
Bit Vector / BitSet	boolean bits	BitSet, Enum collections	compact storage
Skip List	layered sorted list	ConcurrentSkipListMap, ConcurrentSkipListSet	sorted + concurrency

Circular Buffer	ring array	can appear in internal queue impls	efficient FIFO
Array of Segments	split array blocks	ConcurrentHashMap (older), modern uses bins + CAS	concurrency scaling
Immutable structural sharing	persistent structure	List.of(), Set.of(), Map.of() internal impls	memory safe, immutable

---

2) Collections Framework Overview (Hierarchy)

Interface	Implementations
List	ArrayList, LinkedList, Vector, Stack, CopyOnWriteArrayList
Set	HashSet, LinkedHashSet, TreeSet, CopyOnWriteArraySet, EnumSet
Queue	PriorityQueue, ArrayDeque, LinkedList
Deque	ArrayDeque, LinkedList, ConcurrentLinkedDeque
Map	HashMap, LinkedHashMap, TreeMap, Hashtable, ConcurrentHashMap, WeakHashMap, IdentityHashMap, EnumMap, Properties, ConcurrentSkipListMap

---

3) Main Differences Between All Collection Classes (When to Use What)

A) LIST Implementations

Class	Internal DS	Ordering	Duplicates	Null allowed?	Thread-safe?	Best Use Case
ArrayList	Dynamic Array	✅ insertion order	✅	✅	❌	Most common list, fast random access
LinkedList	Doubly Linked List	✅	✅	✅	❌	Many insert/delete in middle, Queue+Deque

Vector	Dynamic Array	✅	✅	✅	✅ (sync)	Legacy, rarely use
Stack	Vector + LIFO	✅	✅	✅	✅	Legacy stack, prefer ArrayDeque
CopyOn WriteArray List	Copy array per write	✅	✅	✅	✅	Many readers, very few writers

---

B) SET Implementations

Class	Internal DS	Ordering	Duplicates	Null allowed?	Thread-safe?	Best Use Case
HashSet	HashMap buckets	❌	❌	✅ (1 null)	❌	Fast membership check
LinkedHash Set	HashSet + linked nodes	✅ insertion order	❌	✅ (1 null)	❌	Unique elements + order needed
TreeSet	Red-Black Tree	✅ sorted	❌	❌	❌	Need sorted unique elements
CopyOn Write ArraySet	CopyOnWriteArrayList	✅	❌	✅	✅	Very read-heavy sets
EnumSet	Bit vector	✅ enum order	❌	❌	❌	Efficient for enum values

---

C) QUEUE / DEQUE Implementations

Class	Internal DS	Ordering	Null allowed?	Thread-safe?	Best Use Case
PriorityQueue	Binary Heap	Priority order	❌	❌	Task scheduling, min/max retrieval

ArrayDeque	Resizable Array Deque	FIFO/LIFO	✕	✕	Best Stack + Queue replacement
ConcurrentLinkedQueue	Lock-free linked nodes	FIFO	✕	✓	Highly concurrent queue
ConcurrentLinkedDeque	Lock-free linked nodes	both ends	✕	✓	Concurrent deque
LinkedBlockingQueue	Linked nodes	FIFO	✕	✓	Producer-consumer (blocking)
ArrayBlockingQueue	Circular array	FIFO	✕	✓	Fixed-size bounded queue
PriorityBlockingQueue	heap + locking	priority	✕	✓	concurrent priority scheduling
DelayQueue	heap + delays	delay	✕	✓	delayed job execution
SynchronousQueue	no storage	handoff	✕	✓	thread-to-thread handoff

---

D) MAP Implementations

Class	**Internal DS**	**Ordering**	**Null allowed?**	**Thread-safe?**	**Best Use Case**
HashMap	buckets + treeify	✕	✓ 1 null key	✕	most common key-value map
LinkedHashMap	HashMap + linked entries	✓ insertion/ access	✓	✕	LRU cache, ordered map
TreeMap	Red-Black Tree	✓ sorted by key	✕	✕	Sorted map
Hashtable	synchronized hash table	✕	✕	✓	Legacy

ConcurrentHash Map	lock-free bins + CAS	✕	✕	✓	best concurrent map
WeakHashMap	weak references	✕	✓	✕	cache with auto cleanup
IdentityHashMap	== instead of equals	✕	✓	✕	special reference-key map
EnumMap	array indexed by enum ordinal	✓ enum order	✕	✕	fastest map for enum keys
Properties	Hashtable based	✕	✕	✓	config .properties files
ConcurrentSkip ListMap	Skip list	✓ sorted	✕	✓	concurrent sorted map

---

4) Java Example Code for Every Individual Collection Class

✓ Below examples are minimal but complete and runnable.

LIST Examples

1) ArrayList

```
import java.util.*;

public class ArrayListExample {
   public static void main(String[] args) {
      List<String> list = new ArrayList<>();
      list.add("Java");
      list.add("Python");
      list.add("Java"); // duplicates allowed

      System.out.println(list);
      System.out.println("Element at index 1: " + list.get(1));
   }
}
```

2) LinkedList

```
import java.util.*;
```

```
public class LinkedListExample {
   public static void main(String[] args) {
      LinkedList<Integer> list = new LinkedList<>();
      list.add(10);
      list.addFirst(5);
      list.addLast(20);

      System.out.println(list);
      list.removeFirst();
      System.out.println("After removeFirst: " + list);
   }
}
```

3) Vector

```
import java.util.*;

public class VectorExample {
   public static void main(String[] args) {
      Vector<String> v = new Vector<>();
      v.add("A");
      v.add("B");
      v.add("C");
      System.out.println(v);
   }
}
```

4) Stack

```
import java.util.*;

public class StackExample {
   public static void main(String[] args) {
      Stack<Integer> stack = new Stack<>();
      stack.push(1);
      stack.push(2);
      stack.push(3);

      System.out.println(stack);
      System.out.println("pop: " + stack.pop());
      System.out.println("peek: " + stack.peek());
   }
}
```

5) CopyOnWriteArrayList

```
import java.util.concurrent.CopyOnWriteArrayList;

public class CopyOnWriteArrayListExample {
```

```
    public static void main(String[] args) {
        CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
        list.add("A");
        list.add("B");

        for (String s : list) {
            list.add("X"); // safe during iteration
        }

        System.out.println(list);
    }
}
```

SET Examples

6) HashSet

```
import java.util.*;

public class HashSetExample {
    public static void main(String[] args) {
        Set<String> set = new HashSet<>();
        set.add("A");
        set.add("B");
        set.add("A"); // ignored

        System.out.println(set);
    }
}
```

7) LinkedHashSet

```
import java.util.*;

public class LinkedHashSetExample {
    public static void main(String[] args) {
        Set<Integer> set = new LinkedHashSet<>();
        set.add(3);
        set.add(1);
        set.add(2);

        System.out.println(set); // keeps insertion order
    }
}
```

8) TreeSet

```
import java.util.*;
```

```
public class TreeSetExample {
   public static void main(String[] args) {
      Set<Integer> set = new TreeSet<>();
      set.add(5);
      set.add(1);
      set.add(3);

      System.out.println(set); // sorted
   }
}
```

9) CopyOnWriteArraySet

```
import java.util.concurrent.CopyOnWriteArraySet;

public class CopyOnWriteArraySetExample {
   public static void main(String[] args) {
      CopyOnWriteArraySet<String> set = new CopyOnWriteArraySet<>();
      set.add("A");
      set.add("B");
      set.add("A");

      System.out.println(set);
   }
}
```

10) EnumSet

```
import java.util.*;

public class EnumSetExample {
   enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

   public static void main(String[] args) {
      EnumSet<Day> workingDays = EnumSet.range(Day.MON, Day.FRI);
      System.out.println(workingDays);
   }
}
```

QUEUE / DEQUE Examples

11) PriorityQueue

```
import java.util.*;

public class PriorityQueueExample {
   public static void main(String[] args) {
```

```
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.add(30);
        pq.add(10);
        pq.add(20);

        while (!pq.isEmpty()) {
            System.out.println(pq.poll()); // 10 20 30
        }
    }
}
```

12) ArrayDeque

```
import java.util.*;

public class ArrayDequeExample {
    public static void main(String[] args) {
        ArrayDeque<String> deque = new ArrayDeque<>();
        deque.add("A");
        deque.addFirst("B");
        deque.addLast("C");

        System.out.println(deque);
        System.out.println("removeFirst: " + deque.removeFirst());
    }
}
```

13) ConcurrentLinkedQueue

```
import java.util.concurrent.ConcurrentLinkedQueue;

public class ConcurrentLinkedQueueExample {
    public static void main(String[] args) {
        ConcurrentLinkedQueue<Integer> q = new ConcurrentLinkedQueue<>();
        q.offer(1);
        q.offer(2);
        q.offer(3);

        System.out.println(q.poll());
        System.out.println(q);
    }
}
```

14) LinkedBlockingQueue

```
import java.util.concurrent.*;

public class LinkedBlockingQueueExample {
    public static void main(String[] args) throws Exception {
```

```
        LinkedBlockingQueue<String> q = new LinkedBlockingQueue<>();
        q.put("Task1");
        q.put("Task2");

        System.out.println(q.take());
        System.out.println(q.take());
    }
}
```

15) ArrayBlockingQueue

```
import java.util.concurrent.*;

public class ArrayBlockingQueueExample {
    public static void main(String[] args) throws Exception {
        ArrayBlockingQueue<Integer> q = new ArrayBlockingQueue<>(2);
        q.put(10);
        q.put(20);

        System.out.println(q.take());
        System.out.println(q.take());
    }
}
```

MAP Examples

16) HashMap

```
import java.util.*;

public class HashMapExample {
    public static void main(String[] args) {
        Map<Integer, String> map = new HashMap<>();
        map.put(1, "A");
        map.put(2, "B");
        map.put(null, "NULL_KEY");

        System.out.println(map);
    }
}
```

17) LinkedHashMap

```
import java.util.*;

public class LinkedHashMapExample {
    public static void main(String[] args) {
        LinkedHashMap<Integer, String> map = new LinkedHashMap<>();
```

```
        map.put(3, "C");
        map.put(1, "A");
        map.put(2, "B");

        System.out.println(map); // keeps insertion order
    }
}
```

18) TreeMap

```
import java.util.*;

public class TreeMapExample {
    public static void main(String[] args) {
        TreeMap<Integer, String> map = new TreeMap<>();
        map.put(3, "C");
        map.put(1, "A");
        map.put(2, "B");

        System.out.println(map); // sorted by key
    }
}
```

19) Hashtable

```
import java.util.*;

public class HashtableExample {
    public static void main(String[] args) {
        Hashtable<Integer, String> map = new Hashtable<>();
        map.put(1, "A");
        map.put(2, "B");

        System.out.println(map);
    }
}
```

20) ConcurrentHashMap

```
import java.util.concurrent.ConcurrentHashMap;

public class ConcurrentHashMapExample {
    public static void main(String[] args) {
        ConcurrentHashMap<Integer, String> map = new ConcurrentHashMap<>();
        map.put(1, "A");
        map.put(2, "B");

        System.out.println(map);
    }
```

```
}
```

21) WeakHashMap

```
import java.util.*;

public class WeakHashMapExample {
   public static void main(String[] args) {
      WeakHashMap<Object, String> map = new WeakHashMap<>();
      Object key = new Object();
      map.put(key, "value");

      System.out.println("Before GC: " + map);
      key = null;
      System.gc();

      System.out.println("After GC: " + map);
   }
}
```

22) IdentityHashMap

```
import java.util.*;

public class IdentityHashMapExample {
   public static void main(String[] args) {
      IdentityHashMap<String, Integer> map = new IdentityHashMap<>();

      String a1 = new String("A");
      String a2 = new String("A");

      map.put(a1, 1);
      map.put(a2, 2);

      System.out.println(map); // both stored because key comparison is ==
   }
}
```

23) EnumMap

```
import java.util.*;

public class EnumMapExample {
   enum Level { LOW, MEDIUM, HIGH }

   public static void main(String[] args) {
      EnumMap<Level, String> map = new EnumMap<>(Level.class);
```

```
        map.put(Level.LOW, "L");
        map.put(Level.HIGH, "H");

        System.out.println(map);
    }
}
```

24) Properties

```
import java.util.*;

public class PropertiesExample {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.setProperty("db.url", "localhost");
        props.setProperty("db.user", "root");

        System.out.println(props.getProperty("db.url"));
        System.out.println(props);
    }
}
```

25) ConcurrentSkipListMap

```
import java.util.concurrent.*;

public class ConcurrentSkipListMapExample {
    public static void main(String[] args) {
        ConcurrentSkipListMap<Integer, String> map = new ConcurrentSkipListMap<>();
        map.put(3, "C");
        map.put(1, "A");
        map.put(2, "B");

        System.out.println(map); // sorted keys
    }
}
```

---

5) Quick Real-Time Selection Guide (One Table)

Requirement	**Best Choice**
Fast random access list	ArrayList
insert/delete middle frequently	LinkedList
thread-safe list with many readers	CopyOnWriteArrayList

unique elements fast lookup	HashSet
unique + insertion order	LinkedHashSet
unique + sorted	TreeSet
queue FIFO	ArrayDeque
priority tasks	PriorityQueue
key-value general	HashMap
key-value sorted	TreeMap
key-value insertion order or LRU	LinkedHashMap
thread-safe key-value	ConcurrentHashMap
memory-sensitive cache keys	WeakHashMap

If you want, I can also provide:

✅ **All classes' Time Complexity table (O(1), O(log n), etc.)**
✅ **Internal working diagrams (HashMap bucket + treeify)**
✅ **Interview questions + answers on Collections Framework**

Sure ✅ Below is the **most important comparison** between:

 HashSet vs LinkedHashSet vs HashMap vs TreeMap
including:

✅ Differences table
✅ Internal implementation
✅ When to use what (real projects)
✅ Java code examples for each
✅ Practical use-case examples

✅ 1) Main Differences (Comparison Table)

Feature	HashSet	LinkedHashSet	HashMap	TreeMap
Type	Set	Set	Map	Map
Stores	Only **unique values**	Only **unique values**	**key → value** pairs	**key → value** pairs

Duplicate allowed?	❌ No	❌ No	❌ No duplicate keys (values can repeat)	❌ No duplicate keys
Ordering	❌ No order	✅ Maintains insertion order	❌ No order	✅ Sorted order by key
Sorting	❌ No	❌ No	❌ No	✅ Yes
Null allowed?	✅ 1 null value	✅ 1 null value	✅ 1 null key + many null values	❌ null keys not allowed
Performance	✅ Fastest average O(1)	Slightly slower than HashSet	✅ Fast average O(1)	Slower O (log n)
Internal DS	HashMap keys	HashMap + doubly linked list	Hash table buckets	Red-Black Tree
Best Use	Fast membership check	Unique + order required	Fast key lookup	Sorted keys required

---

✅ 2) Internals (How they work internally)

Class	Internal Implementation
HashSet	internally uses a HashMap<E,Object> (element stored as key, dummy value)
LinkedHashSet	internally uses LinkedHashMap<E,Object> (keys + linked list for order)
HashMap	hash table buckets + collision handling (Java 8+: treeify to RedBlackTree)
TreeMap	Red-Black Tree (self-balancing BST)

✅ 3) When to Use What (Real-Time Use Cases)

You Need...	Use This
Unique values + fastest search	✅ HashSet
Unique values + insertion order	✅ LinkedHashSet
Key-value storage + fastest lookup	✅ HashMap
Key-value storage + sorted keys	✅ TreeMap

✅ 4) Java Example Code for Each Class

✅ A) HashSet Example

Use Case: Remove duplicates quickly

```
import java.util.*;

public class HashSetDemo {
   public static void main(String[] args) {
      HashSet<String> set = new HashSet<>();

      set.add("Java");
      set.add("Python");
      set.add("Java"); // duplicate ignored
      set.add(null);   // allowed (only one)

      System.out.println("HashSet: " + set);
   }
}
```

 Output order is **random**.

✅ B) LinkedHashSet Example

Use Case: Remove duplicates but preserve input order

```
import java.util.*;

public class LinkedHashSetDemo {
   public static void main(String[] args) {
      LinkedHashSet<String> set = new LinkedHashSet<>();

      set.add("Java");
      set.add("Python");
      set.add("Java"); // ignored
      set.add("C++");

      System.out.println("LinkedHashSet: " + set);
   }
}
```

✅ Output will be:

[Java, Python, C++]

✅ C) HashMap Example

Use Case: Store student roll no -> name

```
import java.util.*;

public class HashMapDemo {
   public static void main(String[] args) {
      HashMap<Integer, String> map = new HashMap<>();

      map.put(101, "Anurag");
      map.put(102, "Sakshi");
      map.put(103, "Rahul");

      map.put(102, "Sakshi Updated"); // overwrites value

      map.put(null, "NULL_KEY_ALLOWED"); // null key allowed
      map.put(104, null); // null value allowed

      System.out.println("HashMap: " + map);

      System.out.println("Value for 101: " + map.get(101));
   }
}
```

 Order is **not guaranteed**.

✅ D) TreeMap Example

Use Case: Sorted key storage

```
import java.util.*;

public class TreeMapDemo {
   public static void main(String[] args) {
      TreeMap<Integer, String> map = new TreeMap<>();

      map.put(3, "C");
      map.put(1, "A");
      map.put(2, "B");

      System.out.println("TreeMap: " + map);
   }
}
```

✅ Output is always sorted:

{1=A, 2=B, 3=C}

✅ 5) Practical Real-Time Use Case Code Examples (When to use what)

✅ Use HashSet: Email validation / Unique emails

```
import java.util.*;

public class UniqueEmails {
   public static void main(String[] args) {
      List<String> emails = Arrays.asList(
            "a@gmail.com", "b@gmail.com", "a@gmail.com"
      );

      Set<String> uniqueEmails = new HashSet<>(emails);

      System.out.println("Unique emails: " + uniqueEmails);
   }
}
```

✅ Best because: fast + removes duplicates

✅ Use LinkedHashSet: Unique but preserve order (form inputs)

```
import java.util.*;

public class PreserveOrderUnique {
   public static void main(String[] args) {
      List<String> items = Arrays.asList("Pen", "Book", "Pen", "Laptop");

      Set<String> result = new LinkedHashSet<>(items);

      System.out.println(result); // [Pen, Book, Laptop]
   }
}
```

✅ Best because: keeps insertion order

✅ Use HashMap: Fast lookup (UserID → Name)

```java
import java.util.*;

public class UserLookup {
   public static void main(String[] args) {
      Map<Integer, String> users = new HashMap<>();
      users.put(1, "Admin");
      users.put(2, "Manager");

      int loginId = 2;
      System.out.println("Logged in user: " + users.get(loginId));
   }
}
```

✅ Best because: O(1) average

✅ Use TreeMap: Sorted reports (Marks sorted by roll number)

```java
import java.util.*;

public class SortedReport {
   public static void main(String[] args) {
      Map<Integer, Integer> marks = new TreeMap<>();
      marks.put(103, 85);
      marks.put(101, 92);
      marks.put(102, 88);

      System.out.println("Sorted marks report: " + marks);
   }
}
```

✅ Best because: output is always sorted

✅ 6) Quick Shortcut Summary (Interview-ready)

Class	1-line summary
HashSet	fastest unique elements storage (unordered)
LinkedHashSet	unique elements + insertion order
HashMap	fastest key-value lookup (unordered)
TreeMap	sorted key-value lookup

If you want, I can also make a **single combined table** including:
✅ HashSet vs TreeSet vs LinkedHashSet and
✅ HashMap vs TreeMap vs LinkedHashMap vs ConcurrentHashMap with complexity + use cases.